

PRODUCT REQUIREMENTS DOCUMENT

OBJECTIVE:

Develop a modern parking system where:

1. Drivers should be able to see the available parking slots before entry.
2. The intended system records vehicles on arrival.
3. On exit the system automatically calculates the total time spent and amount to pay at the exit which in turn enhances the drivers experience

BUSINESS LOGIC

The business logic behind this project depends on the requirements that we were given by our client which include:

1. Drivers should be able to see the available parking slots before entry:

The business logic behind this requirement is that we want to enhance the customer experience of our product by showing them the number of available parking slots since our main objective as a business is to sell a service (parking space) and the business rule is that we cannot sell a space we do not have.

2. The intended system records vehicles on arrival:

This establishes a verifiable starting point for the service rendered. When a vehicle is recorded the business essentially opens a contract with the driver of the car.

3. On exit the system automatically calculates the total time spent and amount to pay at exit:

This is where the transactions comes into place as the business exchanges its physical space over time for money and on exit the barrier opens once payment parking fees is dealt with.

USER FLOW AND USE CASES

USER FLOW:

Registration: The user creates a profile and pays any required Registration Fee. The system saves their details in a Python dictionary.

Entry Request: The user arrives at the lot. The system cross-references their details against a set to ensure there are no duplicate number plate entries or ID numbers.

Space Assignment: The system checks the parking lot dictionary to find an unassigned space among the named spaces (e.g., A1-A30).

Parking: The user parks their vehicle. The system modifies the parking lot dictionary to link the user's data dictionary to that specific space index.

Checkout: The user leaves. The database calculates the total parked hours and processes the final billing.

Release Space: The space's status is cleared, adding to the total number of free spaces once again.

USER CASE

Actor: Registered User (Driver)

Goal: Successfully park a vehicle, track the duration, and pay the corresponding hourly fee.

Preconditions: The user must be registered in the system by providing their User Name, User Age, User ID, User Email, and optionally, their User Vehicle Type.

Step 1: Arrival & Identification: The system identifies the user logging in via their unique assigned ID, such as "ABC-232".

Step 2: Space Allocation: The system checks the Parking Lot Data to determine the total number of free spaces. It locates a spot (e.g., "A1") and updates its status to "occupied", assigning it to the specific user.

Step 3: Duration Tracking: The system records the entry time to determine how long a certain user has parked for.

Step 4: Exit & Billing: Upon exit, the system applies the fees to charge per hour to effectively bill the user the right amount based on hours parked.

CONSTRAINTS

Immutability of Tuples: Tuples cannot be modified once they are created, meaning the system cannot use them to add new users dynamically.

Duplicate Value Rejection in Sets: Sets inherently reject duplicate values. If two users happen to have the exact same name, the system would fail to store both, which forces the reliance on strictly unique identifiers like ID numbers or number plates.

Dictionary Efficiency: While dictionaries are noted as the best choice for mapping parking spaces, they are not considered the most efficient data structure for handling multiple broad entries overall.

Data Mapping Dependencies: The system relies entirely on unique keys to function; a single user must be accurately mapped to a list of payments and a specific parking index to effectively calculate billing.

Data Structures

This document evaluates four main structures, ultimately selecting three to act as the "tiny database":

- **Dictionaries (The Core Database):** This is the primary structure used because of its key-value mapping capabilities.
 - **User Profiles:** Maps a unique identifier to user details (e.g., `User_dic= {"ABC-232": {"name": "James Omuri", "age": 20}}`).
 - **Lot Management:** Maps a specific space to its occupancy status and the assigned user (e.g., `"A1": {"status": "occupied", "user-assigned-to": James}}`).

- **Billing Records:** Maps a single user to their payment amounts (e.g., {"Abdul": 300}).
- **Lists (The Containers):** Selected because they are mutable (modifiable) once created. A master list (e.g., User_List) is used to hold all the individual user dictionaries or sets.
- **Sets (The Validators):** Selected specifically for their inability to hold duplicate values. They are utilized to store critical unique data, preventing duplicate ID numbers or vehicle number plates from entering the system.
- (Note: Tuples were explicitly disqualified because they cannot be modified, making it impossible to add new users.)

Algorithms and System Logic

While the text does not contain mathematical algorithms, it clearly defines the logical algorithms the system must execute:

- **State Verification Algorithm:** The system must scan the parking_Lot structure to calculate the total number of free spaces versus occupied spaces, determining exactly which space is vacant.
- **Assignment & State Mutation:** Once a vacant space is identified, the system algorithm modifies the dictionary, changing the status from "free" to "occupied" and inserting the user's data dictionary into that space's index.
- **Authentication & Retrieval:** The system uses unique keys (like "ABC-232") to instantly determine which user has logged in and retrieve their nested data (like age or name).
- **Billing Algorithm (The Assignment):** The document tasks you with figuring out the billing logic. The implied algorithm would require tracking the entry time, determining how long the user parked, and multiplying that duration by the "Fees to Charge per hour," alongside any "Registration Fee".

